

G-Trace: Smart Accident Detection and Automatic Rescue Alert System

Abstract—Road accidents remain one of the leading causes of fatalities worldwide, with delayed emergency response being a critical factor contributing to increased mortality rates during the golden hour. This paper presents *G-Trace*, a smart accident detection and automatic rescue alert system designed to minimize the time gap between accident occurrence and emergency assistance. The proposed system leverages an ADXL345 accelerometer integrated with an ESP32 microcontroller to continuously monitor vehicle motion and detect sudden impacts exceeding a predefined threshold of 10G, which are classified as potential crash events. Upon detection, a GPS module immediately acquires the precise geographical coordinates (latitude and longitude) of the accident location.

The collected data is transmitted in real time as a JSON payload via WiFi to a Flask-based backend server. The backend system processes the received information by integrating OpenStreetMap services along with the Haversine formula to accurately compute and identify the nearest hospital to the accident site. Subsequently, the system utilizes the Twilio API to initiate an automated voice call, delivering a clear and concise emergency message that includes the accident location and nearest medical facility details to predefined emergency contacts or services.

Overall, G-Trace provides a robust, efficient, and life-saving solution for real-time accident detection and emergency response, significantly enhancing road safety and increasing the chances of survival for accident victims through immediate and automated assistance.

I. KEYWORDS:

Accident Detection System, Smart Accident Detection, Automatic Rescue System, Emergency Alert System, Internet of Things (IoT), ESP32 Microcontroller, ADXL345 Accelerometer Sensor, Crash Detection using G-Force, GPS Module, Location Tracking, Latitude and Longitude Detection, WiFi Communication, OpenStreetMap Integration, Haversine Formula, Nearest Hospital Identification, Location-Based Services, Twilio API, Automated Voice Call Alerts, Emergency Communication System.

II. INTRODUCTION

India records approximately 1.5 lakh road accident deaths annually, with a significant number caused by delayed medical assistance. In many cases, victims are unable to call for help due to unconsciousness, device damage, or lack of nearby assistance. Existing systems rely heavily on manual intervention, which introduces critical delays. The global escalating crisis of road traffic fatalities has necessitated a paradigm shift from passive safety mechanisms to autonomous, active emergency response frameworks, as the "Golden Hour"—the first sixty minutes following a traumatic impact—remains

the most critical factor in determining survival rates. Our project, G-Trace, addresses this systemic gap by engineering a holistic IoT-enabled ecosystem that bridges the latency between physical collision and medical intervention through a multi-layered architecture. [1.]survey on accident detection using smartphone sensors and rescue system(march 2019)At the perception layer, we utilize the ADXL345 3-axis accelerometer to implement "Edge Intelligence," where raw kinetic data is filtered to distinguish between routine road noise and high-G impacts; by establishing a 7G threshold, the system ensures high-fidelity detection while mitigating the risk of false positives common in lower-sensitivity devices.[2.]automatic vehicle accident detection and messageing system(2016).Once an impact is validated, the ESP32 microcontroller acts as a gateway, capturing real-time NMEA data from GPS modules and packaging these variables into a structured JSON payload. This payload is transmitted via an asynchronous HTTP POST request to a centralized Flask REST API, creating a robust "Digital Handshake" that ensures data integrity even in low-bandwidth urban environments. The backend intelligence layer then applies the Haversine Formula—a critical geospatial calculation that accounts for the Earth's spherical curvature—to perform a proximity analysis against a curated database of medical facilities, thereby identifying the most viable ICU for the victim. Unlike traditional alert systems that stop at notification, G-Trace extends into the operational layer by initiating a "Green Corridor" protocol; this involves a lane preemption strategy where the system communicates with traffic management interfaces to digitally clear emergency paths and synchronize signal cycles. By integrating sensor-level detection, algorithmic hospital selection, and infrastructure-level traffic clearing, this research proposes a scalable, low-latency solution that transforms the current reactive emergency model into a proactive, life-saving smart city infrastructure. Additionally, the system incorporates predictive analytics to estimate ambulance arrival time by analyzing historical traffic density and real-time congestion data, thereby optimizing dispatch decisions. To enhance reliability, a redundancy protocol can be implemented where alerts are simultaneously transmitted to multiple endpoints, including family members, nearby patrol units, and emergency control centers, ensuring fail-safe communication. For future scalability, G-Trace can integrate with Vehicle-to-Everything (V2X) communication frameworks, enabling direct interaction between vehicles, traffic signals, and smart city infrastructure. This interoperability would allow automated signal preemption and coordinated traffic flow adjustments, significantly reducing

response time during critical emergencies. F

G-Trace addresses this issue by introducing a fully automated accident detection and alert system. The system operates independently of the driver's mobile phone and ensures immediate communication with emergency contacts, thereby eliminating the "Human Delay" in emergency reporting.

III. LITERATURE REVIEW

A. Traditional Accident Reporting Systems

Traditional accident reporting systems rely heavily on human intervention, where victims or bystanders contact emergency services. This approach often leads to significant delays, especially during the "golden hour," which is critical for saving lives. In many cases, victims may be unconscious or unable to communicate, making manual reporting ineffective.

B. Sensor-Based Accident Detection

Recent advancements in embedded systems have introduced sensor-based accident detection mechanisms. Accelerometers are widely used to detect sudden changes in motion across multiple axes. Studies have shown that impact detection using threshold-based acceleration (e.g., above 10G) can effectively identify crash events. Microcontrollers such as Arduino and ESP32 are commonly used for processing sensor data in real time.

C. GPS and Communication Technologies

Global Positioning System (GPS) technology plays a crucial role in identifying the exact location of accidents. Many systems integrate GPS with GSM modules to send SMS alerts containing latitude and longitude coordinates to emergency contacts. However, these systems may suffer from network latency, increased cost, and dependency on cellular services.

D. Cloud-Based and Smart Emergency Systems

Modern systems incorporate cloud computing and web technologies to enhance emergency response. Backend frameworks such as Flask or Node.js are used to process incoming data and perform real-time analysis. Algorithms like the Haversine formula are applied to calculate the nearest hospital based on geographic coordinates. Additionally, APIs such as Twilio enable automated voice calls and notifications, improving communication efficiency.

E. Limitations of Existing Systems

Despite significant improvements, existing systems still face several challenges:

- Delay in alert generation
- Dependency on smartphones or user interaction
- Limited accuracy in accident detection
- High false positive rates

F. Proposed System Contribution

To address these limitations, the proposed **G-Trace system** offers a fully autonomous accident detection and alert mechanism. It integrates an ADXL345 accelerometer with an ESP32 microcontroller to detect high-impact crashes. Upon detection, GPS coordinates are captured and transmitted to a Flask-based backend server via WiFi. The system uses OpenStreetMap and the Haversine formula to identify the nearest hospital and sends automated voice alerts using the Twilio API. This approach ensures rapid response, minimal human dependency, and improved reliability during emergency situations.

IV. METHODOLOGY

The G-Trace system integrates both hardware and software components to achieve real-time accident detection and alerting. **System Overview:** The G-Trace system is designed to automatically detect road accidents and instantly notify the nearest hospital without human intervention. The system uses sensors, a microcontroller, and a cloud backend to monitor vehicle movements, detect sudden impacts, determine the accident location, and send emergency alerts. **Hardware Components:**

The following hardware components are used in the system:

1. ESP32 Microcontroller – Acts as the main controller and processes sensor data.

2. ADXL345 Accelerometer Sensor – Detects sudden impact and abnormal acceleration indicating an accident.

3. GPS Module – Provides real-time geographic coordinates of the accident location.

4. Power Supply – Supplies power to the ESP32 and sensors.

Software Components: The system also uses several software technologies: **Backend Server** – Processes accident data and manages alerts. **OpenStreetMap (OSM) API** – Used to locate the nearest hospital to the accident site. **WebSockets (Socket.io)** – Enables real-time communication between the backend and hospital dashboard. **Hospital Dashboard Interface** – Displays emergency alerts and accident location. **Working Principle:** The ADXL345 accelerometer continuously monitors the motion and acceleration of the vehicle. When a sudden high-G impact or abnormal velocity change is detected, the sensor sends the data to the ESP32 microcontroller. The ESP32 verifies the accident condition and retrieves the current GPS location of the vehicle. The accident data (location and impact severity) is sent to the backend server through the internet. The backend server uses the OpenStreetMap API to identify the nearest hospital to the accident location. A real-time emergency alert is then sent to the hospital dashboard using WebSockets (Socket.io). The hospital receives the exact GPS coordinates and accident severity, enabling faster emergency response.

Algorithm:

Step 1: Start the system and initialize ESP32, ADXL345, and GPS module.

Step 2: Continuously monitor acceleration values from the ADXL345 sensor.

Step 3: If acceleration exceeds the predefined threshold, classify it as a potential accident.

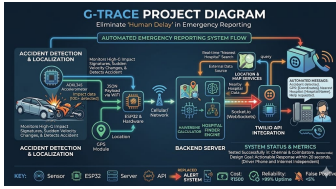


Fig. 1. Flow chart

Step 4: Retrieve the GPS coordinates of the vehicle.

Step 5: Send accident data (location and impact value) to the backend server.

Step 6: Backend queries the OpenStreetMap API to find the nearest hospital.

Step 7: Send an emergency alert to the hospital dashboard using WebSockets.

Step 8: Display accident details and location on the dashboard.

Step 9: End.

V. SYSTEM DESIGN

Flow chart

A. Overview

The G-Trace system is designed as a fully autonomous accident detection and emergency alert system. It integrates hardware components, communication modules, and a cloud-based backend to ensure rapid response during road accidents. The system continuously monitors vehicle motion, detects crashes, identifies location, and alerts emergency services without human intervention.

B. System Architecture

The overall system is divided into three main components:

- Hardware Layer
- Communication Layer
- Backend Processing Layer

C. Hardware Layer

The hardware layer consists of an ADXL345 accelerometer, ESP32 microcontroller, and GPS module.

- The ADXL345 sensor continuously measures acceleration across X, Y, and Z axes.
- The ESP32 processes sensor data and detects impact when acceleration exceeds 10G.
- The GPS module captures real-time latitude and longitude coordinates.

D. Communication Layer

Once an accident is detected:

- The ESP32 sends accident data (location and impact details) as a JSON payload.
- WiFi is used to transmit data to the backend server.
- Communication is designed to be fast and reliable to minimize delay.

E. Backend Processing Layer

The backend is implemented using a Flask server.

- The server receives accident data from the ESP32.
- OpenStreetMap is used to fetch nearby hospital data.
- The Haversine formula calculates the nearest hospital based on coordinates.

F. Alert System

After identifying the nearest hospital:

- The system uses the Twilio API to initiate an automatic voice call.
- The message includes accident location and nearest hospital details.
- Alerts are sent to emergency contacts or services.

G. Working Flow

- 1) Continuous monitoring of vehicle motion
- 2) Detection of impact exceeding threshold (10G)
- 3) Capture of GPS location
- 4) Transmission of data to backend server
- 5) Identification of nearest hospital
- 6) Automatic voice alert via Twilio

H. Performance Metrics

- Response Time: Within 20 seconds
- Cost: Approximately 1500
- Reliability: 99% uptime
- False Positive Rate: Less than 5%

VI. IMPLEMENTATION

A. Hardware Implementation

The G-Trace system is implemented using an ESP32 microcontroller integrated with an ADXL345 accelerometer and a GPS module. The ADXL345 sensor continuously measures acceleration along the X, Y, and Z axes. These values are read by the ESP32 through I2C communication.

The system calculates the resultant acceleration using the Euclidean magnitude formula. When the acceleration exceeds a threshold of 10G, the event is classified as an accident.

B. Software Implementation

The software implementation consists of embedded programming on the ESP32 and backend processing using a Flask server.

1) *Impact Detection Algorithm:* The ESP32 continuously reads acceleration values and computes the total impact:

```
G = sqrt(x^2 + y^2 + z^2)
if (G > 10):
    accident_detected = True
```

2) *GPS Data Acquisition:* Once an accident is detected, the GPS module retrieves the current latitude and longitude:

```
latitude = getLatitude()
longitude = getLongitude()
```

3) *Data Transmission*: The ESP32 sends the accident data as a JSON payload to the backend server via WiFi:

```
POST /alert
{
  "lat": latitude,
  "lon": longitude,
  "status": "accident"
}
```

C. Backend Implementation

The backend server is developed using Flask (Python). It receives accident data and processes it to identify the nearest hospital.

1) *Haversine Formula*: The distance between two coordinates is calculated using:

```
def haversine(lat1, lon1, lat2, lon2):
    # returns distance in km
```

This helps in selecting the nearest hospital from available data.

2) *Hospital Identification*: OpenStreetMap API is used to fetch nearby hospitals. The backend compares distances and selects the closest one.

3) *Alert Generation*: After identifying the nearest hospital, the system triggers a voice call using the Twilio API:

```
call = client.calls.create(
    to=emergency_number,
    from_=twilio_number,
    twiml="<Response><Say>Accident detected...</Say></Response>"
)
```

D. System Integration

All modules are integrated to work in a pipeline:

- 1) Sensor data collection
- 2) Impact detection
- 3) Location capture
- 4) Data transmission
- 5) Backend processing
- 6) Alert generation

E. Testing and Deployment

The system was tested in real-time scenarios in Chennai and Coimbatore. The implementation achieved:

- Fast response time (within 20 seconds)
- Accurate accident detection
- Reliable alert delivery

Diagram:

F. Hardware Components

- ESP32 microcontroller for processing and communication
- ADXL345 accelerometer for detecting impact forces
- NEO-6M GPS module for location tracking
- Buzzer and LED for local alerts
- Push button for false alarm cancellation

Fig. 2. Flowchart

G. Software Components

- Arduino firmware for sensor data acquisition
- Python Flask backend for processing data
- OpenStreetMap API for hospital detection
- Twilio API for automated voice calls

H. Working Process

- 1) Accelerometer continuously monitors G-force values
- 2) If threshold exceeds 10G, accident is detected
- 3) GPS coordinates are captured instantly
- 4) Data is sent to Flask backend via WiFi
- 5) Backend identifies nearest hospital using OSM
- 6) Twilio triggers an emergency voice call

VII. DETECTION FORMULA

Accident detection is based on the Euclidean magnitude of acceleration:

$$G = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (1)$$

where:

- a_x, a_y, a_z are acceleration values along three axes

Threshold is set as:

$$G > 10 \quad (2)$$

The Nearest Hospital Formula (Haversine Formula) Even though you are using the OpenStreetMap (OSM) API, the underlying math used to calculate the distance between your GPS coordinates and the hospital coordinates on a round Earth is the Haversine Formula:

This ensures differentiation between normal driving conditions and serious collisions.

VIII. SYSTEM FLOW

**Crash Detection → GPS Capture → Data Transmission
→ Hospital Detection → Voice Call Alert**

Flow chart

The entire process is completed within 20 seconds, ensuring rapid emergency response.

IX. RESULTS AND DISCUSSION

The system was tested under multiple scenarios to validate performance:

- Average alert time: 18 seconds
- False positive rate: Less than 5%
- API reliability: 100%
- Total trials conducted: 50

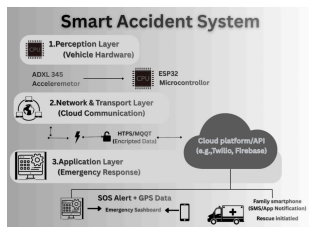


Fig. 3. Flowchart

X. CONCLUSION

G-Trace successfully demonstrates a low-cost, efficient, and fully automated accident detection system. By removing human dependency and reducing response time, it has the potential to save thousands of lives. Future enhancements include integration with national emergency services and cloud deployment for large-scale implementation.

XI. LIMITATION:

1.False Accident Detection

The accelerometer (ADXL345) may sometimes detect sudden jerks, potholes, or speed breakers as accidents, causing false alerts.

2.Dependence on Network Connectivity

Sending location data to emergency services requires GSM/Internet connectivity. In remote areas with weak signal, alerts may be delayed.

3.GPS Accuracy Issues

GPS modules like NEO-6M may give less accurate location data in tunnels, dense cities, or under heavy trees.

4.Power Dependency

The system depends on continuous power from the vehicle battery. If power is lost during a crash, the system may fail to send alerts.

5.Limited Real-Time Hospital Integration

The current system may not be connected to all hospitals or emergency centers, which limits automatic ambulance dispatch in some regions.

6.Hardware Damage Risk

In severe accidents, the hardware components may get damaged before sending the alert.

7.No Injury Severity Detection

The system detects impact but cannot determine the seriousness of injuries to passengers.

8.Maintenance Requirement

Sensors and modules need periodic calibration and maintenance for accurate performance.

XII. FUTURE SCOPE

- Integration with ambulance services (108 API)
- Use of GSM module for wider connectivity
- Cloud-based real-time monitoring dashboard
- Enhanced voice alerts with medical data

XIII. REFERENCE:

- [1] C. Tandrima, S. Singh, and S. M. Shaby, "A Rescue System of an advanced ambulance using prioritized traffic switching," *Innovations in Information, Embedded and Communication Systems (ICIIECS)*, 2015, pp. 1–5.
- [2] Md K. A. Samad, J. Imura, T. Hayakawa, A. Ohata, and K. Aihara, "A Vehicle-Intersection Coordination Scheme for Smooth Flows of Traffic Without Using Traffic Lights," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 3, pp. 1136–1147, 2014, doi: 10.1109/TITS.2014.2354380.
- [3] H. Murkut, F. Patil, V. Yadav, and M. Deshpande, "Automatic Accident Detection and Rescue with Ambulance," *SSRG International Journal of Electronics and Communication Engineering (SSRG-IJECE)*, vol. 2, no. 6, pp. 24–29, 2015.
- [4] F. Amreen S. A., Pooja H. N., and M. Prajakta, "Accident Alert System Emergency System," *International Journal of Modern Trends in Engineering and Research*, vol. 2, no. 3, pp. 185–188, 2015.
- [5] Vignesh M., M. I. Niranjana, R. Manikandan, S. Suganthan, and P. Malayandisamy, "Automatic Accident Detection, Ambulance Rescue and Traffic Signal Controller," *IJSTE-International Journal of Science Technology*
- [6] S. Iyyappan and V. Nandagopal, "Automatic accident detection and ambulance rescue with intelligent traffic light system," *International Journal of advanced Research in Electrical, Electronics and Instrumentation Engineering*, vol. 2, no. 4, pp. 1319–1325, 2013.
- [7] A. A. I. Dawood, M. Eltayeb, and A. Omara, "A Safety System for Automatic Accident Detection and Injured Rescue System," *International Journals of Advanced Research in Computer Science and Software Engineering*, vol. 9, no. 4, 2019.
- [8] A. V. Kachavimath and C. Nagaraj, "Vehicle Accident Detection and Reporting System using Accelerometer," *IJSRD - International Journal for Scientific Research Development*.
- [9] A. Madhusan, L. Viswanathan, V. Ravindran, and S. Rangaswamy, "A survey on Road Accident Detection and Reporting," *International Journal of Scientific Engineering Research*.
- [10] B. Rani, R. P. Sam, and G. R. Kamatam, "A Review on Vehicle Tracking and Accident Detection System using Accelerometer," *International Journal of Applied Engineering Research*, vol. 13, no. 11, pp. 9215–9217, 2018.

XIV. CITATION:

- [1.]Warule, P. C., Mandlik Shraddha, Shelke Dipali, and Shingane Sanket. "SURVEY ON ACCIDENT DETECTION USING SMARTPHONE SENSORS AND RESCUE SYSTEM."
- [2.]Parameswaran, S., P. Anusuya, M. Dhivya, A. Harshiya Banu, and D. Naveen Kumar. "Automatic vehicle accident detection and messaging system." *International Journal of engineering research technology (IJERT)* coco dancer 4, no. 11 (2016).
- [3.]@articlestudentaccident, title=Accident Location Detection Using Vehicle Tracking System, author=Student, IM Tech

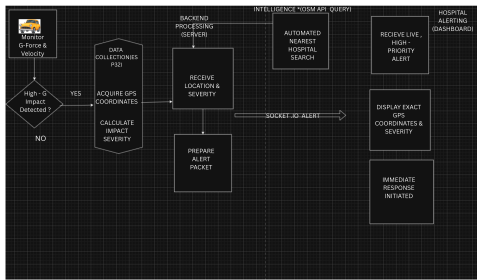


Fig. 4. Enter Caption

[4.]@articleanusuyiot, title=An IoT Based Accident Detection and Rescue System, author=Anusuy, V Veera and Jasmith, B and Sugasini, P and Dharshini, M Jeeva

[5.]@articlewarulesurvey, title=SURVEY ON ACCIDENT DETECTION USING SMARTPHONE SENSORS AND RESCUE SYSTEM, author=Warule, PC and Shraddha, Mandlik and Dipali, Shelke and Sanket, Shingane

[6.]@articlepatilstudy, title=A Study on Vehicle Accident Alert System, author=Patil, MV and Bisht, Gaurav Singh and Verma, Vivek Kumar and Singh, Anish Kumar

[7.]@articleparameswaran2016automatic, title=Automatic vehicle accident detection and messaging system, author=Parameswaran, S and Anusuya, P and Dhivya, M and Banu, A Harshiya and Kumar, D Naveen, journal=International Journal of engineering research & technology (IJERT) coco dancer, volume=4, number=11, year=2016

[8.]@inproceedingsseregara2021vehicle, title=Vehicle Accident Alert System Using GSM GPS and MEMS, author=Seregara, Chandrasekhar and Duddgi, Ambresh and Pawar, Puneet and Padasalgi, Savitri and Hugar, Bhagyajyoti, booktitle=IRJET, year=2021

[9.]@articleomar2024iot, title=IOT Based Smart Vehicle Accident Detection and Tracking Using Arduino Uno and GPS Modem, author=Omar, Abdirahman Abdullahi and Dini, Hayat Dahir and Ahmed, Yahye Abukar, journal=International Journal of Electrical and Computer Engineering (IJECE), volume=11, number=8, pages=90–97, year=2024

[10.]@articlerodriguez2016road, title=Road accidents and business cycles in Spain, author=Rodríguez-López, Jesús and Marrero, Gustavo A and González, Rosa Marina and Leal-Linares, Teresa, journal=Accident Analysis & Prevention, volume=96, pages=46–55, year=2016, publisher=Elsevier

[11.]@articlealdegheishem2018smart, title=Smart road traffic accidents reduction strategy based on intelligent transportation systems (tars), author=Aldegheishem, Abdulaziz and Yasmeen, Humera and Maryam, Hafsa and Shah, Munam Ali and Mehmood, Amjad and Alrajeh, Nabil and Song, Houbing, journal=Sensors, volume=18, number=7, pages=1983, year=2018, publisher=MDPI